

Catan Game Project: Code Overview

Anthony van Tilburg

Introduction

This overview (version 4), crafted as of May 2026, highlights the project's core components and GUI. It marks a significant new milestone: full human-player interactivity during Initial Placement, combined with a rich selection of placement algorithms available for all AI players.

The project initializes an empty game board consisting of 45 tiles (19 land tiles and 26 sea tiles), a detailed scoreboard displaying player statistics, and an interaction panel with action buttons such as "Buy City" and "Roll Dice". The GUI is divided into three main areas:

- The **playboard**, which visually represents the hexagonal board with tiles, intersections, ports, and number chits;
- The **scoreboard**, showing each player's resources and key game statistics;
- The **button panel**, which enables the human player to interact with the game.

Versions 1 and 2 focused on establishing the visual foundation and core game logic. Version 3 introduced full human interactivity during Initial Placement, while the current version (v4) expands this further by implementing multiple placement algorithms for AI players.

Core Modules

The project's structure, as outlined in Annex 1, is organized into directories that separate concerns effectively.

Core

The core/ directory houses the essential game logic, comprising four key modules:

- board.py,
- player.py
- game.py
- constants.py

Each module plays a critical role in initializing and managing the game's foundational elements.

The board.py module is responsible for creating and managing the game board. It defines the layout of 45 tiles (19 land + 26 sea tiles, along with intersections and roads). The board can be initialized with a predefined configuration or randomly generated, ensuring flexibility for different game setups. It also handles port placement.

The player.py module manages player data, including resources (wheat, ore, wood, brick and sheep), development cards, and game statistics like victory points. It supports both human and AI players, with attributes initialized to reflect an empty starting state. This module is pivotal for tracking player progress, which is displayed on the scoreboard.

The game.py module serves as the central hub, orchestrating game state, player management, and board interactions. It initializes the game with a sequence number, ID and phase (e.g., "Initial Placement"), setting up players and the board. It also includes supporting classes for tracking turn details and resource distribution, ensuring all game elements are synchronized.

The constants.py module defines game-wide constants, such as resource costs (e.g., 1 wheat, 1 wood, 1 brick, 1 wool for a settlement) and log file paths. It also provides utility functions for calculations, ensuring consistency across modules.

Gui

The gui/ directory houses the essential GUI logic, now comprising five key modules:

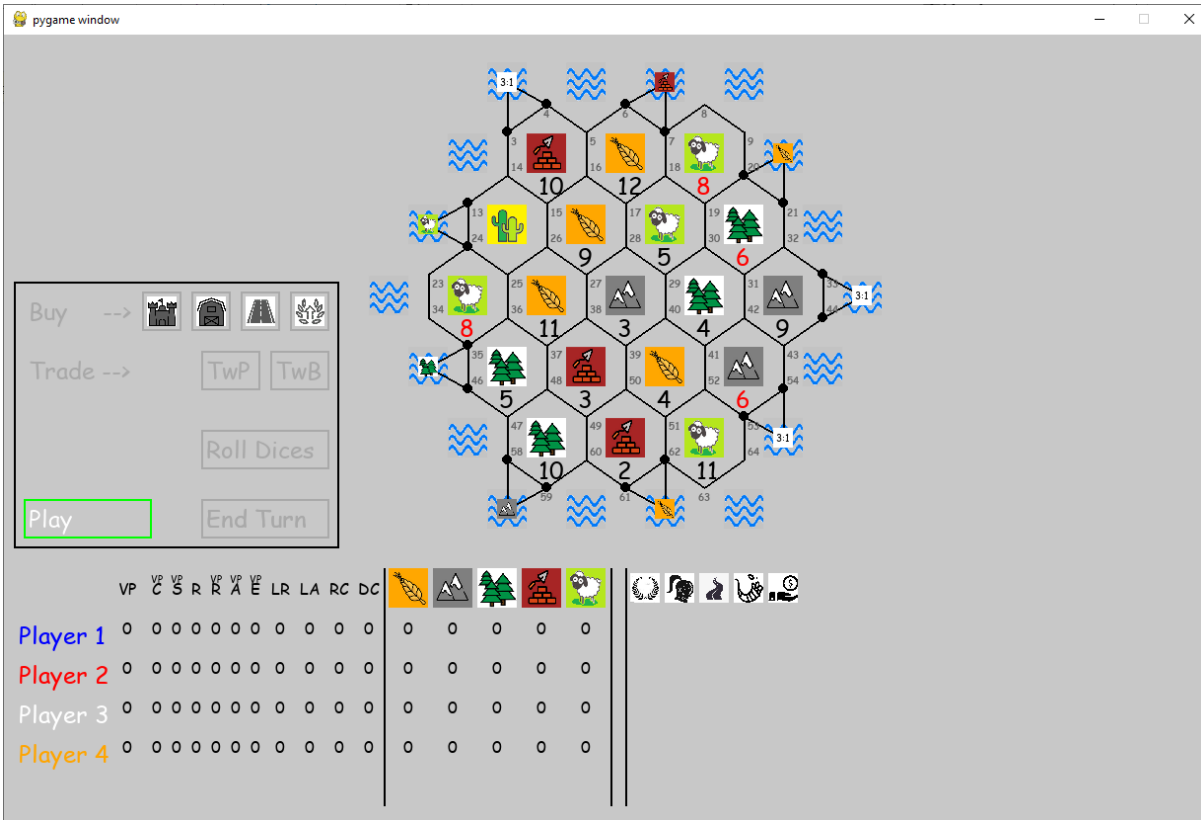
- gui.py (playboard rendering, animation queue, scoreboard, and general GUI management)
- gui_guidance.py (state machine for human placement)
- gui_human_player.py (button rendering and mode handling)
- gui_constants.py (positioning constants, fonts, colours, images, sounds)
- event_handler.py (mouse-click processing and routing)

Each module plays a critical role in rendering the game and enabling smooth human interaction.

Module	Description
GUI	Manages board rendering, the central animation queue, scoreboard updates, and overall GUI orchestration.
HumanGuidance	Implements the complete state machine for human initial placement (CHOOSING_SETTLEMENT → SETTLEMENT_SELECTED → CHOOSING_ROAD → ROAD_SELECTED), including pulsing highlights and on-screen OKY/OKN confirmation icons.
GUIHumanPlayer	Handles rendering of all action buttons (“Buy City”, “Roll Dice”, “Play”, etc.) and their active/inactive states based on game phase and current mode.
GUI Constants	Centralizes all positioning constants, fonts, colours, images, and sounds used throughout the GUI.
EventHandler	Processes every mouse click and routes it correctly (first to confirmation icons, then to the “Play” button, then to HumanGuidance).

Note: gui_game.py functionality (in version 1) has been integrated directly into gui.py for streamlined updates.

These core modules form the foundation of the game’s logic, enabling the GUI to visualize the initial state effectively; example of the gui down below.



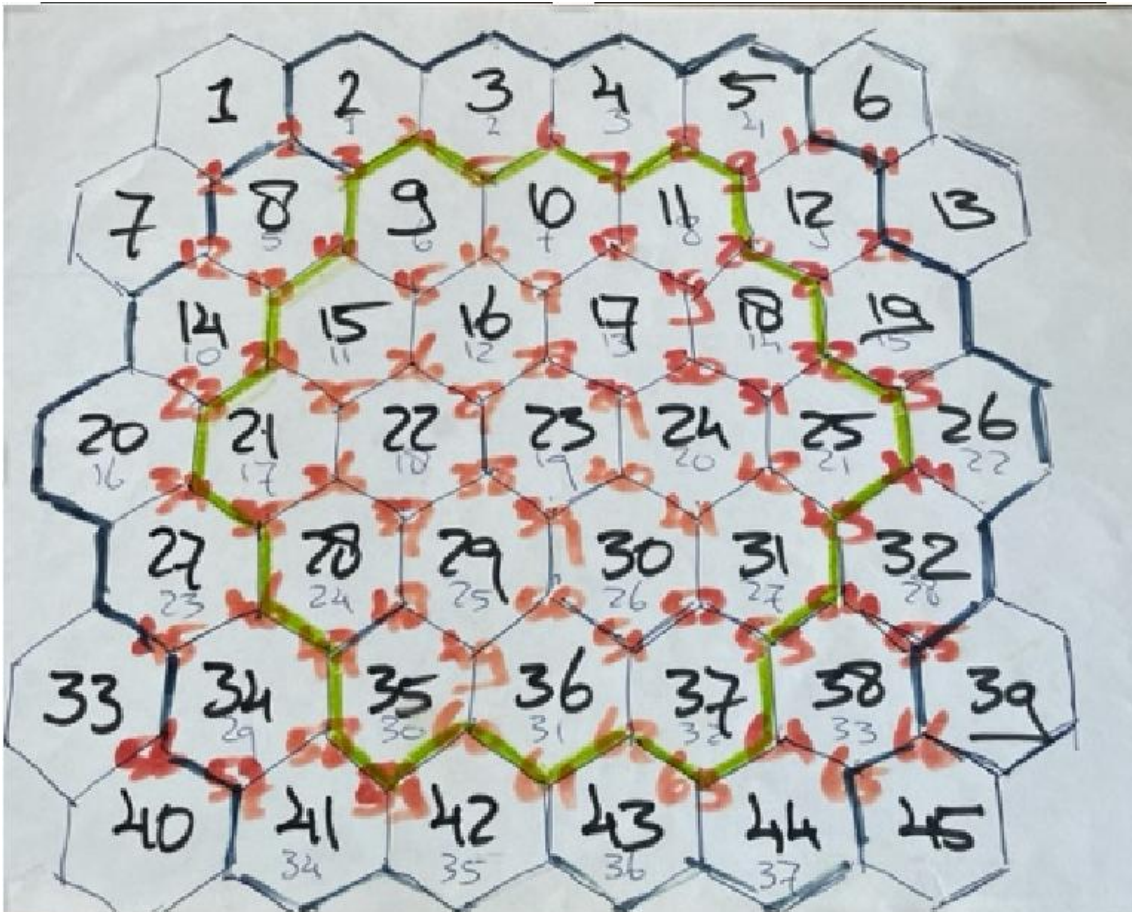
Core Classes

The project's functionality is encapsulated in 20 classes, as listed in Annex2, which define the game's entities and their interactions. Below, we highlight the core classes critical to this first version, focusing on their roles in initializing the game and supporting the GUI.

Core

The Game class (game.py) is the central orchestrator, managing the game's state, including the phase ("Initial Placement"), turn, and player list. It initializes the board and players, setting up the initial empty state. Supporting classes like StrategyDashboard, TurnDetails, ResourceCardDashboard, and Settings track player statistics, turn-specific details, resource distribution, and game configurations, respectively. These classes ensure that the game state is robust and ready for future gameplay mechanics.

The Board class (board.py) defines the game board, managing 45 tiles through the Tile class, 66 intersections via the Intersection class, and 68 roads via the Road class. The Tile class handles tile properties (e.g. type,value), Intersection tracks vertex states (e.g., occupancy, port status), and Road manages edge connections between Tiles. These classes enable the playboard's visual representation ensuring proper setup.



- Tiles: 0, 1, 6, 7, 13, 33, 39, 40, 45 have type None.
- Intersections: 0, 1, 2, 10, 11, 12, 22, 45, 55, 56, 57, 65, 66 have canbuild_ynx = "X"
- Roads: (1,2), (2,3), (9,10), (10,11), (12,13), (21,22), (45,46), (54,55), (56,57), (57,58), (64,65), (65,66) have type None.

The Player class (player.py) represents a player, managing resources, development cards, settlements, cities, and roads. It initializes with zero resources and tracks statistics like victory points, which are displayed on the scoreboard. The ResourceAsResourceCard enum (constants.py) defines resource types (e.g., WHEAT, ORE), ensuring consistent resource management.

Gui

The HumanGuidance class (gui_guidance.py) – state controlling, pulsing highlights, and confirmation icons. Supported state changes:

- CHOOSING_SETTLEMENT → SETTLEMENT_SELECTED (via on_board_click after valid intersection click)
- SETTLEMENT_SELECTED → CHOOSING_ROAD (via on_confirmation OKY: settlement placed)
- CHOOSING_ROAD → ROAD_SELECTED (via on_board_click after valid road click)
- ROAD_SELECTED → IDLE (via on_confirmation OKY + _finish_human_turn: road placed + "Play" button enabled)

The EventHandler class (event_handler.py) – processes all mouse clicks and delegates to guidance or the "Play" button.

The GUI class (gui.py) - central rendering engine that draws the static board base, all permanent settlements/roads/cities, manages the animation queue for pulsing highlights and quarter-circle reveal

animations, updates the scoreboard and round/turn display, handles screenshots, and integrates the HumanGuidance system for interactive initial placement.

The GUIHumanPlayer class (`gui_human_player.py`) – renders and controls all interaction-panel buttons (Buy City, Buy Settlement, Buy Road, Buy DCard, TwP, TwB, Roll Dices, End Turn, Cancel, and the key “Play” button) with dynamic active/inactive states based on game phase and current HumanGuidance state.

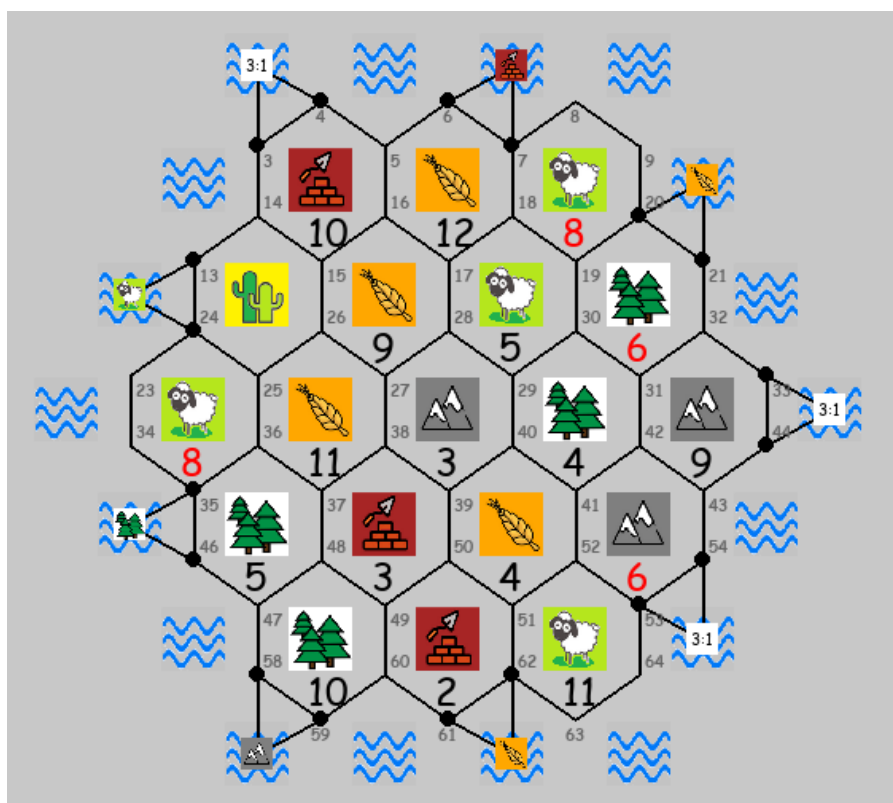
The Button class (`gui.py`) tracks button states for the interaction panel. These classes, detailed in Annex 2, are the backbone of the project’s initial state.

GUI Appearance

The GUI is a cornerstone of the first and second version, designed to visually represent the game’s state through three distinct parts: the playboard, scoreboard, and buttons for interaction.

The playboard, rendered by the GUI class using methods like `display_fresh_board()` and `_draw_tiles()`, displays the 45-tile board with hexagonal tiles, intersections, ports, and number chits.

Tiles are drawn with images (e.g., `Field.png`) from the Image enum, with red chits for high-probability values (6, 8) and black for others. All intersections are uniquely numbered (`intersection.id`). The first intersection on the playboard is numbered: 3. Dual ports are highlighted with black dots and show their type via a small image.



The scoreboard, managed by `update_scoreboard()`, displays player statistics (VP, C, S, R, LR, LA, RC, DC) with coloured player names, resource images (40×40), and development-card images (30×30). Vertical lines separate Statistics / Resource Cards / Development Cards.

	VP	C	S	R	R	A	E	LR	LA	RC	DC									
Player 1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
Player 2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
Player 3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
Player 4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

VP = VictoryPoints

C = Number of Cities on playboard (max 4)

S = Number of Settlements on playboard (max 5)

R = Number of Settlements on playboard (max 15)

R = Extra VPs due to Longest Route (max 2)

A = Extra VPs due to Largest Army (max 2)

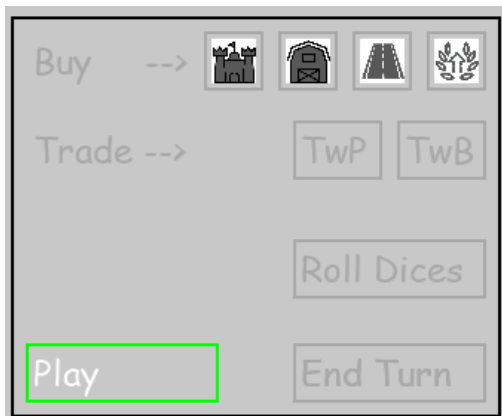
E = Extra VPs due to drawn VPs from Development Cards (max 5)

LR = Longest Route for specific player (example: 2)

LA = Largest Army for specific player (example: 0)

RC = Number of Resource Cards in players hand

DC = Number of Development Cards in players hand



The interaction panel managed by xxx shows:

- “Buy →”, “Trade →”, “Roll Dices”, “End Turn”, and “Play” buttons.

The panel is drawn with a black border. Button states are controlled by `set_button()` / `check_button()` and logged.

Added functionality since the first version:

- Dynamic guidance text (e.g., “Click a highlighted intersection for your settlement”).
- Pulsing coloured circles on every available settlement/road spot (via the central animation queue).
- OKY / OKN confirmation icons that appear exactly on the chosen location after a click.

Initial Placement

Initial Placement is guided by a flexible algorithm system implemented at the player level. The project now supports multiple placement strategies, giving both AI players and the human player (via simulation) access to different levels of sophistication.

Architecture

- **initial_placement.py** — Main orchestration module. It coordinates which algorithm to use based on settings and executes the placement logic.
- **algorithms_initial_placement.py** — Contains the actual implementation of all available algorithms.

Available Algorithms

1. **_max_pips** Places the settlement on the intersection with the highest total pip value (probability of resource production).
2. **_max_pips_and_ports** Extends the above by also considering proximity to favorable ports (especially 2:1 ports matching high-yield resources).
3. **_five_strategy_engine** A more advanced rule-based system using weighted combinations of multiple factors:
 - Balanced
 - Wood/Brick focused
 - Wheat/Ore focused
 - Wheat/Ore/Wool focused
 - Monopoly / High-probability focused
4. **Markov Chain Evaluator** The most sophisticated algorithm. It uses Markov Chain modeling to evaluate long-term expected value of each possible starting position.

Markov Evaluator Module

The markov_evaluator.py module was added as a separate file because it contains a significantly more complex implementation involving:

- Transition probability matrices
- Dynamic programming for efficient state value calculation
- Expected value computation across multiple future turns

This separation keeps the code clean and maintainable — the core logic remains isolated from the simpler heuristic algorithms.

The Markov approach was inspired by [Lauren Nagels' paper: "Analysis of 'The Settlers of Catan' Using Markov Chains"](#) (Texas Christian University, 2020s). We enhanced the original concept with dynamic programming techniques to make it computationally feasible for real-time use during initial placement.

Initial Placement is guided by an algorithm set on a player level. AI players use the original logic; the human player uses a fully interactive flow.

AI players (or when HUMAN_PLAYER=False):

1. Position settlement at intersection with highest pips.
2. Position road toward a convenient port (3:1 or 2:1 if ≥ 3 pips of that resource).
3. Immediate placement + GUI update.

Human player (when it is the human's turn):

1. Guidance starts in CHOOSING_SETTLEMENT – all legal intersections pulse.
2. Click a highlighted intersection → SETTLEMENT_SELECTED; OKY/OKN icons appear.
3. OKY → settlement placed (resources distributed if round == -1).
4. Automatically switches to CHOOSING_ROAD – connected roads pulse.
5. Click a road → ROAD_SELECTED + new OKY/OKN icons.
6. OKY → road placed.
7. “Play” button becomes active; clicking it advances to the next player.

Additional features (unchanged for both AI and human):

- Second settlement (round -1) triggers `distribute_initial_resources`.
- Road selection uses BFS (`_find_nearest_port`). [BFS = Breath-First Search](#).
- Every placement triggers GUI update with animation (settlement + road + optional resource-tile highlights).